

Apostila

Boas Práticas no Laboratório de Bioinformática

Organização • Terminal • SSH • Ambientes • Git • Reprodutibilidade

Objetivo: estabelecer hábitos de trabalho que tornem análises bioinformáticas organizadas, seguras, reproduzíveis e fáceis de compartilhar.

1. Por que boas práticas importam?

Em bioinformática, um experimento raramente termina quando um programa produz um resultado. É comum precisar, semanas ou meses depois, responder: de onde veio esse arquivo? Qual versão do programa foi utilizada? Quais parâmetros foram usados? Quem executou a análise? O resultado pode ser reproduzido?

“Regra geral: outra pessoa — ou você mesmo daqui a seis meses — deve conseguir entender o que foi feito sem precisar perguntar ao autor.”

2. Organização de arquivos e pastas

Evite trabalhar diretamente em Downloads ou Desktop. Prefira uma estrutura específica para cada projeto.

```
projeto_cancer_mama/  
├── README.md  
├── data/  
├── results/  
├── scripts/  
├── notebooks/  
├── figures/  
├── docs/  
└── environment/
```

Uma estrutura comum em bioinformática separa dados originais, dados processados e resultados:

```
projeto_genomica/  
├── README.md  
├── data/  
│   ├── raw/  
│   └── processed/  
├── scripts/  
├── notebooks/  
├── results/  
├── figures/  
├── logs/  
└── environment/
```

Princípio: dados brutos → processamento → resultados. Evite misturar tudo em uma única pasta.

3. Nomes de arquivos e pastas

Use letras minúsculas, números quando necessários, _ ou - como separador e nomes descritivos. Evite espaços e caracteres especiais.

```
rna_seq_analysis/  
sample_001.fastq.gz  
gene_expression.csv  
quality_control.py
```

Evite nomes ambíguos como dados.csv, novo.csv, final.csv ou script2.py. O nome deve indicar o que o arquivo contém ou faz.

Para datas, prefira YYYY-MM-DD, por exemplo 2026-08-17. Esse formato também mantém os arquivos em ordem cronológica.

4. Dados brutos não devem ser alterados

Mantenha os arquivos originais em data/raw/. Se precisar transformar um arquivo, gere um novo arquivo em data/processed/.

```
data/raw/sample01.fastq.gz  
↓  
data/processed/sample01_trimmed.fastq.gz
```

“Nunca modifique o arquivo original para facilitar a análise. Preserve a origem dos dados.”

5. Variáveis e funções

Escolha nomes que expliquem o significado da informação. Em Python, `snake_case` é uma convenção comum.

```
sample_name = "sample_01"
read_count = 150000
reference_genome = "hg38"

def filter_low_quality_reads():
    ...
```

Evite `x`, `y`, `z`, `calc()` e `process()` quando nomes mais específicos forem possíveis. Comentários devem explicar principalmente por que algo é feito, e não apenas repetir o código.

6. Terminal: pense antes de executar

Alguns comandos podem modificar ou apagar muitos arquivos rapidamente. Tenha atenção especial com `rm`, `mv`, `cp`, `chmod`, `chown` e `sudo`.

```
pwd
ls
```

Antes de uma operação destrutiva, confirme onde está e o que existe naquele diretório. Não use `sudo` apenas para contornar um erro sem entender sua causa.

7. Ambientes virtuais

Projetos diferentes podem depender de versões diferentes de Python e de bibliotecas. Ambientes virtuais isolam essas dependências.

```
python -m venv .venv

# Linux
source .venv/bin/activate

# Windows PowerShell
.venv\Scripts\Activate.ps1

python -m pip install pandas numpy
pip freeze > requirements.txt

deactivate
```

A pasta `.venv/` não deve ser versionada no Git. Em bioinformática, Conda, Mamba e Micromamba também são muito utilizados, especialmente quando existem dependências além de Python.

8. Git e versionamento

Git registra a evolução do código e permite voltar a versões anteriores. Em vez de `script_v2.py`, `script_final.py` e `script_final_REAL.py`, use versionamento.

Normalmente devem ser versionados: `scripts`, `src`, `notebooks`, `README.md`, `requirements.txt`, `environment.yml`, `.gitignore`, documentação e testes.

Normalmente não devem ser versionados: `.venv/`, `__pycache__`, arquivos temporários, logs enormes, dados brutos gigantes, resultados gigantes, senhas, tokens e chaves privadas.

```
.venv/
__pycache__/
*.pyc
*.log
.env
```

Commits devem representar mudanças lógicas. Exemplos: Add quality control script, Fix sample ID parsing, Update README with installation instructions.

9. README e documentação

Todo projeto relevante deve ter um README.md explicando o objetivo, requisitos, instalação, execução, localização dos dados e principais decisões.

```
# Gene Expression Analysis

## Objetivo
Analisar a expressão diferencial entre os grupos.

## Requisitos
- Python 3.12
- pandas
- numpy

## Instalação
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

## Execução
python scripts/run_analysis.py

## Estrutura
- data/raw: dados originais
- data/processed: dados processados
- results: resultados
- scripts: scripts de análise
```

Também registre versões de ferramentas, parâmetros, referências usadas e decisões relevantes. Não dependa da memória.

10. SSH e segurança básica

SSH permite acessar máquinas remotamente. Nunca compartilhe sua senha, chave privada, tokens ou arquivos de credenciais.

```
ssh usuario@servidor
whoami
hostname
pwd
```

Ao trabalhar com vários servidores, confirme quem você é, em qual máquina está e em qual diretório está antes de executar comandos importantes.

Não execute cegamente comandos encontrados na internet, especialmente comandos que baixam e executam scripts ou usam sudo. Leia, entenda e verifique a fonte.

11. Dados científicos e segurança

Dados de pesquisa podem conter informações identificáveis ou clínicas. Antes de copiar, compartilhar ou enviar dados para serviços externos, verifique as políticas do projeto e da instituição.

“Nunca envie dados sensíveis para serviços externos de IA, armazenamento ou análise sem verificar se isso é permitido.”

12. Caminhos e portabilidade

Evite caminhos específicos da sua máquina, como /home/jose/projeto/dados.csv ou C:\Users\Jose\. Prefira caminhos relativos ou construídos pelo programa.

```
from pathlib import Path

project_dir = Path(__file__).resolve().parent.parent
data_file = project_dir / "data" / "raw" / "sample.csv"
```

Isso torna o projeto mais fácil de mover para outro computador ou servidor.

13. Configuração e segredos

Separe configurações do código quando isso facilitar a reprodução. Um arquivo YAML pode concentrar parâmetros.

```
threads: 16
reference: "hg38"
quality_threshold: 20
```

Nunca coloque senhas, tokens ou chaves privadas diretamente no código ou no Git. Variáveis de ambiente e mecanismos de gerenciamento de segredos são preferíveis.

14. Automatização e reprodutibilidade

Quanto mais importante e repetitiva for uma análise, menos ela deve depender de operações manuais. Scripts e workflows tornam o processo mais consistente.

Ferramentas de workflow comuns em bioinformática incluem Snakemake, Nextflow, WDL e CWL. Não é necessário dominar todas imediatamente.

Uma análise é mais reprodutível quando outra pessoa consegue repetir o resultado usando dados + código + versões + parâmetros + ambiente.

15. Notebooks

Jupyter Notebook é excelente para exploração, visualização e prototipagem, mas análises importantes não devem depender de executar células manualmente em uma ordem específica.

Antes de compartilhar um notebook: reinicie o kernel, execute tudo do início ao fim e verifique se os resultados são reproduzidos.

16. Teste em pequena escala

Antes de processar centenas de gigabytes, teste com uma amostra ou subconjunto pequeno. Verifique resultado, tempo, memória, armazenamento, parâmetros e erros.

“Teste pequeno primeiro. Escale depois.”

17. Monitoramento de recursos

```
top
htop
df -h
du -sh *
```

Antes de uma análise grande, verifique CPU, memória e espaço em disco. Em servidores compartilhados, respeite as regras do laboratório e use o scheduler previsto, como SLURM, quando aplicável.

18. Git não é backup

Git registra versões, mas não substitui uma estratégia de backup. Para dados importantes, uma referência clássica é a regra 3-2-1: 3 cópias, 2 tipos diferentes de mídia e 1 cópia fora do local

principal.

Uma cópia no mesmo disco não protege contra falha do disco, exclusão acidental, corrupção ou ransomware.

19. Integridade de arquivos

Para arquivos transferidos, especialmente arquivos grandes, hashes podem ajudar a verificar a integridade.

```
sha256sum arquivo.fastq.gz
```

Calcule o hash no local de origem e no destino. Valores iguais fornecem forte evidência de que os arquivos são idênticos.

20. Um modelo de projeto

```
project/
├── README.md
├── .gitignore
├── requirements.txt
├── data/
│   ├── raw/
│   └── processed/
├── scripts/
├── notebooks/
├── results/
├── figures/
├── logs/
├── docs/
└── .env/
```

O `.env/` existe localmente, mas não deve ser versionado.

21. Fluxo de trabalho recomendado

1. Crie o projeto e a estrutura de diretórios. 2. Preserve os dados brutos. 3. Crie o ambiente. 4. Instale e registre dependências. 5. Documente objetivo e parâmetros. 6. Teste com uma amostra pequena. 7. Execute a análise. 8. Registre resultados e logs. 9. Versione as alterações. 10. Faça backup dos dados importantes.

22. Checklist diário

- ☐ Os arquivos estão no diretório correto.
- ☐ Os arquivos possuem nomes claros.
- ☐ Dados brutos estão separados dos processados.
- ☐ Não sobrescrevi dados importantes.
- ☐ O código está organizado.
- ☐ Variáveis e funções possuem nomes claros.
- ☐ Dependências estão registradas.
- ☐ O ambiente virtual está separado do código.
- ☐ Não existem senhas ou tokens no repositório.
- ☐ Mudanças importantes foram registradas no Git.
- ☐ O README está atualizado quando necessário.

- [] Parâmetros importantes estão documentados.
- [] Resultados importantes possuem origem rastreável.
- [] Dados importantes possuem backup.

23. Checklist antes de uma análise grande

- [] Estou no servidor correto?
- [] hostname confirma a máquina?
- [] pwd confirma o diretório correto?
- [] Tenho espaço suficiente em disco?
- [] Tenho memória e CPU suficientes?
- [] O ambiente correto está ativado?
- [] As versões das ferramentas estão corretas?
- [] Os dados de entrada estão corretos?
- [] Existe backup dos dados importantes?
- [] Fiz um teste com uma amostra menor?
- [] Sei onde os resultados serão salvos?
- [] Sei como interromper o processo se algo der errado?

24. Seis perguntas para qualquer análise

1. O que estou fazendo? — objetivo da análise.
2. Com quais dados? — entradas e origem.
3. Com qual código? — scripts ou pipeline.
4. Com quais ferramentas? — programas e versões.
5. Com quais parâmetros? — configuração.
6. Como cheguei a este resultado? — histórico e documentação.

25. Princípios para levar para o laboratório

“Organize antes de começar.”

“Não destrua o dado original.”

“Nomeie as coisas de forma clara.”

“Não dependa da memória. Documente.”

“Não dependa do seu computador. Declare o ambiente.”

“Não use sudo sem entender o motivo.”

“Não coloque segredos no Git.”

“Não confunda Git com backup.”

“Teste em pequena escala antes de executar em grande escala.”

“Automatize processos que precisam ser repetidos.”

“Um resultado científico deve ser rastreável até sua origem.”

Objetivo final: outra pessoa deve conseguir receber os dados, código, versões, parâmetros e instruções e reproduzir o que foi feito.